МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 3

Дисциплина: «Аппаратные средства телекоммуникационных систем» Тема: «Умножение матриц. Сравнение последовательной и параллельной версий программы»

Работу выполнил
Студент гр. 4404
Семенов М.А.

Казань, 2026

1. Цель работы

Изучить два подхода к распараллеливанию задач в OpenMP: распараллеливание циклов (data parallelism) и функциональную декомпозицию (task parallelism). Сравнить их работу на примере вычисления геометрического центра облака точек в 3D-пространстве.

2. Задание

Задача 5. Написать программу, которая читает из файла координаты точек в 3D пространстве (x, y, z) и вычисляет геометрический центр (среднее по x, y и z). Написать две версии программы: одну с распараллеливанием цикла, другую с функциональной декомпозицией.

Общие требования:

- Число потоков передается через командную строку.
- Программа должна корректно работать с файлами.
- Замеры времени подтверждаются скриншотами.

3. Ход работы

3.1. Подготовка данных

Для тестирования программы используется текстовый файл points.txt. Чтобы не создавать его вручную, в программу встроен генератор: если файл не найден, программа автоматически создаёт его и заполняет 1 000 000 случайных точек. Это обеспечивает достаточную вычислительную нагрузку для замеров времени.

3.2. Версия 1: Распараллеливание цикла (Data Parallelism)

В первой версии реализован классический подход с распределением итераций цикла между потоками. Чтение массива точек и суммирование координат выполняется в одном цикле. Для безопасного накопления сумм используется директива #pragma omp parallel for с клаузой reduction:

```
#pragma omp parallel for reduction(+:sum_x, sum_y, sum_z) schedule(static)
for (int i = 0; i < N; ++i) {
   sum_x += points[i].x;
   sum_y += points[i].y;
   sum_z += points[i].z;
}
```

Каждый поток накапливает локальные суммы в своих копиях переменных, после чего они безопасно складываются в общую переменную без гонок данных.

3.3. Версия 2: Функциональная декомпозиция (Task Parallelism)

Во второй версии задача разбивается на независимые функциональные блоки (по одной координате на блок). Для этого используется директива #pragma omp parallel sections:

```
#pragma omp parallel sections reduction(+:sum_x, sum_y, sum_z)
{
#pragma omp section
   { for (int i = 0; i < N; ++i) sum_x += points[i].x; }
#pragma omp section
   { for (int i = 0; i < N; ++i) sum_y += points[i].y; }
```

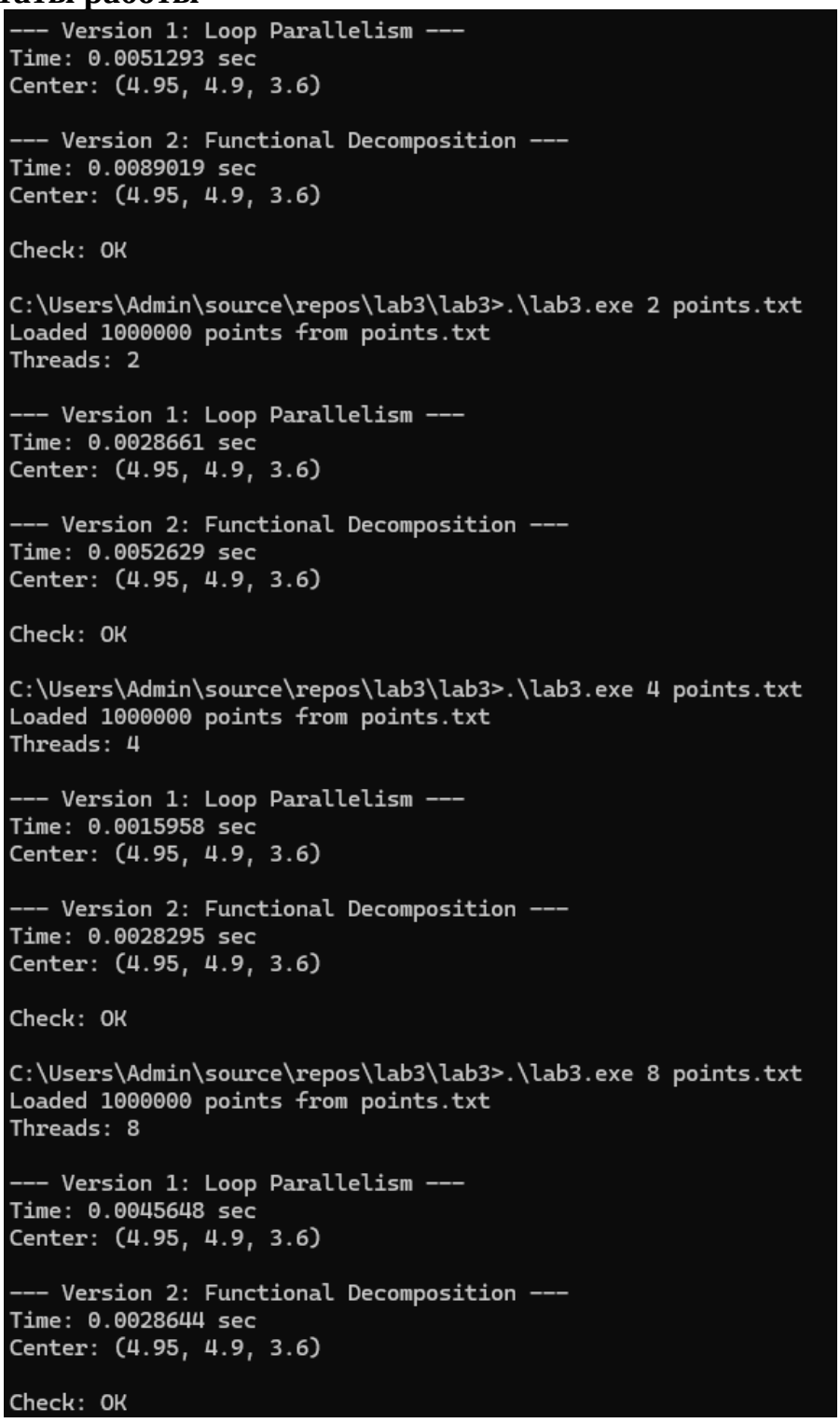
```
#pragma omp section
    { for (int i = 0; i < N; ++i) sum_z += points[i].z; }
}
```

Здесь каждый поток выполняет свой уникальный цикл (считает сумму только по одной оси).

3.4. Проверка корректности

В конце программы координаты центра, вычисленные обеими версиями, сравниваются с помощью допуска. Выводится результат Check: OK, если расхождение минимально.

4. Результаты работы



Сводная таблица измерений (на основе запусков):

Число потоков	Время Версии 1 (Цикл), сек	Время Версии 2 (Секции), сек	Геометрический центр (X, Y, Z)
1	0.005129	0.008901	(4.95, 4.9, 3.6)
2	0.002866	0.005262	(4.95, 4.9, 3.6)
4	0.001595	0.002829	(4.95, 4.9, 3.6)
8	0.004564	0.002864	(4.95, 4.9, 3.6)

5. Анализ результатов

Во всех запусках проверка выдала Check: OK, координаты центра совпадают, что подтверждает корректность обеих стратегий распараллеливания.

Сравнение подходов:

1. **Распараллеливание цикла (**parallel for**)** показывает наилучшую производительность на малом числе потоков. На 4 потоках время сократилось с 0.0051 до 0.0015 сек (ускорение ~3.2x). Это объясняется тем, что 1 миллион итераций отлично делится на любое число потоков, обеспечивая равномерную загрузку процессора.
2. **Функциональная декомпозиция (**sections**)** ожидаемо уступает по абсолютному времени. Поскольку задача разбита всего на 3 независимые секции (сумма по X, Y и Z), при запуске на 4 и более потоках время выполнения перестает уменьшаться (составляет ~0.0028 сек). Остальные потоки простаивают, так как параллельно могут работать только 3 потока (по числу секций).
3. При запуске Версии 1 на 8 потоках наблюдается рост времени по сравнению с 4 потоками (с 0.0015 до 0.0045 сек). Это связано с архитектурой процессора (вероятно, 4 физических ядра и 8 логических), из-за чего потоки конкурируют за ресурсы, а также с накладными расходами OpenMP на синхронизацию (reduction), которые при столь малом времени работы становятся заметными.

---

6. Выводы

В ходе лабораторной работы были изучены две модели параллельного программирования OpenMP:

- **Data parallelism (**#pragma omp parallel for**)** — позволяет быстро и эффективно распараллелить циклы с независимыми итерациями. С помощью клаузы reduction решается проблема безопасного суммирования.
- **Task parallelism (**#pragma omp parallel sections**)** — позволяет распараллелить разнородные блоки кода.

Экспериментально подтверждено, что для обработки больших массивов данных (1 млн точек) гораздо эффективнее использовать распараллеливание цикла, так как оно обеспечивает лучшую масштабируемость и равномерную загрузку всех доступных ядер. Функциональная декомпозиция имеет ограничение по масштабируемости (не более числа логических секций) и подходит для задач, состоящих из разных по своей природе действий.